

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ БЮДЖЕТНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**ФИНАНСОВЫЙ УНИВЕРСИТЕТ
ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ
Липецкий филиал**

Кафедра «Учет и информационные технологии в бизнесе»

Направление: Прикладная математика и информатика

Профиль: Анализ данных

КУРСОВОЙ ПРОЕКТ

по дисциплине: «Машинное обучение»

**ТЕМА: Анализ поведенческих закономерностей пользователей
социальных сетей**

Преподаватель: к.ф.-м.н., доцент Калитвин В. А.

Студент: Шуркалова А. Н.

Личное дело №10016240077

Курс второй

Липецк 2025

Содержание

ВВЕДЕНИЕ	3
1. Общая характеристика алгоритмов и методов машинного обучения	4
1.1. Описание задачи и этапов её решения	4
1.2. Описание моделей машинного обучения, используемых в работе	5
1.2.1. Анализ поведения пользователей в социальных сетях . .	5
1.2.2. Метрики расстояния	7
1.2.3. Алгоритм К-ближайших соседей	9
1.2.4. Алгоритм К-Means	10
1.2.5. Методы оценки качества моделей	12
1.3. Краткая информация об инструментах разработки	14
1.4. Описание источников данных	14
2. Построение моделей машинного обучения и анализ результатов	14
2.1. Получение данных для построения моделей	14
2.2. Предобработка и исследовательский анализ данных	15
2.3. Построение моделей	15
2.3.1. Кластеризация пользователей методом К-Means	15
2.3.2. Классификация пользователей методом KNN	16
2.4. Оценка качества моделей	17
2.4.1. Оценка качества кластеризации	17
2.4.2. Оценка качества классификации	18
2.4.3. Сравнение метрик расстояния	18
2.5. Анализ результатов и рекомендации	19
ЗАКЛЮЧЕНИЕ	21
ПРИЛОЖЕНИЕ	22

ВВЕДЕНИЕ

В настоящее время социальные сети стали неотъемлемой частью нашей жизни. Ежедневно мы оставляем там большие объёмы своих данных, так называемый цифровой след. Эта информация показывает поведенческие паттерны человека, то есть устойчивые модели поведения, поддающиеся выявлению и анализу. Знание этих закономерностей важно как для самих платформ, чтобы улучшить системы рекомендаций и удержать внимание пользователя. Также эта информация нужна и бизнесу, чтобы адаптировать рекламу под интересы читателя.

По некоторым исследованиям выявлено, что использование социальных сетей носит привычный характер, то есть поведение пользователя можно предугадать. Как раз для выявления таких паттернов используются методы машинного обучения, в частности алгоритмы классификации и кластеризации. Классификация — это метод, который относит объект к одному из заранее известных классов (например, «активный пользователь» или «пассивный»). Кластеризация же позволяет группировать объекты без обучающих меток, определяя скрытые закономерности в данных. Также в российской науке активно развиваются методы анализа цифрового следа пользователей на основе данных социальных сетей. Но вопрос о том, какие алгоритмы и метрики расстояния покажут наиболее точные результаты при анализе именно поведенческих паттернов, остаётся открытым.

В данной работе объектом исследования являются социальные сети как среда взаимодействия пользователей и генерация поведенческих данных. Предметом исследования служат поведенческие закономерности пользователей, выраженные в числовых признаках, таких как частота взаимодействия пользователей с контентом, время активности или другие действия, отражающие вовлечённость.

Целью курсовой работы является анализ поведенческих закономерностей пользователей социальных сетей с использованием методов машинного обучения и выявление наиболее подходящего подхода к кластеризации и классификации пользователей на основе их паттернов.

Чтобы достигнуть поставленной цели, понадобится решить следующие задачи:

1. Изучить теоретические основы анализа социальных сетей и методов машинного обучения, а именно метрики расстояния, такие как евклидово, манхэттенское, Чебышёва, и алгоритмы KNN (К-ближайших соседей), K-Means.
2. Рассмотреть, какие есть подходы к анализу поведенческих паттернов пользователей в социальных сетях, и выделить ключевые поведенческие признаки.
3. Найти и подготовить датасет, который будет содержать в себе числовые данные о поведении пользователей, пригодные для применения методов KNN

и K-Means.

4. Реализовать алгоритмы KNN (для классификации) и K-Means (для кластеризации) с использованием языка Python и различных библиотек.

5. Провести эксперименты вычислений: подобрать оптимальные параметры моделей (значение K для KNN, число кластеров для K-Means), оценить качество классификации (точность, полнота, F1-мера) и кластеризации (инерция, силуэт).

6. Истолковать результат, который получится, и сформулировать рекомендации по персонализации контента для различных групп пользователей.

После выполнения работы планируется получить модель, применимую на практике. С её помощью можно будет делить людей на группы по их поведению в соцсетях. Подразумевается, что метод K-Means будет делить людей на несколько групп, с помощью этих групп станет легко выявить активных или пассивных пользователей. А алгоритм KNN, как предполагается, будет правильно определять принадлежность пользователя, то есть достигнет точности классификации как минимум 80% на тестовой выборке.

Курсовая работа состоит из введения, двух глав, заключения и списка литературы. В первой главе рассматриваются теоретические основы анализа социальных сетей и алгоритмов машинного обучения. Во второй главе приводится описание датасета, предобработка данных, реализация моделей, анализ результатов и рекомендации.

1. Общая характеристика алгоритмов и методов машинного обучения

1.1. Описание задачи и этапов её решения

В этой работе мы решаем две задачи: понять, как пользователи ведут себя в соцсетях, и научиться это анализировать с помощью машинного обучения.

Процесс разбит на такие шаги:

- 1) Разобраться, какие данные оставляют пользователи (лайки, комментарии, время в сети, посты) и выделить главные признаки.
- 2) Выбрать методы для работы: KNN для классификации, K-Means для кластеризации. Изучить метрики расстояния и оценки качества.
- 3) Найти подходящий датасет с поведенческими признаками. Подготовить данные: убрать пропуски, привести к одному масштабу.
- 4) Написать код на Python с использованием библиотек sklearn, pandas, matplotlib.

- 5) Оценить, насколько хорошо работают модели, нарисовать графики и сформулировать рекомендации — какой контент подходит разным группам пользователей.

1.2. Описание моделей машинного обучения, используемых в работе

1.2.1. Анализ поведения пользователей в социальных сетях

Данные в социальных сетях

Данные соцсетей делятся на различный контент и связи между пользователями. Контент максимально разнообразный, и для каждой социальной сети есть свои предпочтения:

- с текстовым контентом (Twitter, Telegram);
- с преобладанием изображений (Instagram¹);
- с преобладанием видео (YouTube);
- смешанного типа (ВКонтакте, Facebook).

Зачастую данные соцсетей неструктурированные, и на это стоит обратить внимание, потому что потребуется применение специальных методов обработки, таких как нормализация, стандартизация и выделение числовых признаков, которые в дальнейшем используются в машинном обучении в таких алгоритмах, как KNN и K-Means.

Поведенческие паттерны пользователей

По мнению авторов, анализ социальных сетей изучает поведение отдельных пользователей на микроуровне, общую структуру связей на макроуровне и их взаимодействие [?]. В исследованиях активности пользователей обычно выделяют три типа:

- Активные пользователи — те, кто наиболее часто публикует различный контент, ставит лайки и взаимодействует с другими.
- Средние пользователи — самая распространённая группа, иногда взаимодействуют с другими, реже что-то публикуют, ставят лайки.
- Пассивные пользователи — зарегистрированы на площадке, но почти не проявляют активности.

¹ Социальная сеть Instagram (принадлежит компании Meta, признанной экстремистской и запрещённой на территории Российской Федерации) используется в работе исключительно как объект анализа, без цели пропаганды.

Исследование пользователей в Индонезии выделило два кластера:

- Кластер 0 (активные): пользуются Instagram, TikTok, YouTube по несколько часов в день, очень высокий уровень взаимодействия, интересуются развлекательным и мотивационным контентом.
- Кластер 1 (пассивные): в основном Instagram и TikTok, 3–4 часа в день, низкий уровень вовлечённости, предпочитают мотивационный контент и саморазвитие.

Это подтверждает, что K-Means эффективно и хорошо сегментирует пользователей по поведению.

Поведенческие признаки, используемые для анализа

Чтобы оценить поведение пользователей, используют целевые действия, такие как лайки, репосты, комментарии, переходы по ссылкам. Также учитывается время, проведённое в соцсети. Кроме того, в датасетах для анализа часто применяют демографические признаки (возраст, пол, образование, локацию), метрики вовлечённости (количество подписчиков, постов, время на платформах), а также активность в будние и выходные дни.

Для определения активности пользователей исследователи использовали топологические признаки эго-сетей (структуры личных связей пользователя) и сделали вывод, что активные пользователи показывают более сложную динамику и разнообразные паттерны, чем пассивные.

Задачи анализа поведения пользователей

На основе обзорной статьи по ML для соцсетей авторы отмечают, что соцсети являются одними из самых богатых источников своевременных и полных онлайн-данных. Методы машинного обучения позволяют:

- Сегментировать аудиторию — выделять группы пользователей со схожими паттернами поведения;
- Классифицировать пользователей — определять, относится ли пользователь к активным или пассивным;
- Прогнозировать действия — предсказывать, будет ли пользователь взаимодействовать с контентом;
- Персонализировать контент — адаптировать рекомендации под конкретные группы пользователей.

Из исследования про Telegram авторы выделили 4 типа пользователей в дискуссиях о COVID-19 и изменении климата: Strategic Disruptor, Empirical

Enthusiast, Inquisitive Moderate, Critical Examiner. Классификация достигла точности 0.95–0.99, то есть 95–99%.

Роль машинного обучения в анализе поведения

- Кластеризация (K-Means) применяется, когда неизвестно, какие группы существуют. Алгоритм способен самостоятельно находить скрытые закономерности и объединять пользователей по общим интересам.
- Классификация (KNN) используется, когда есть размеченные примеры (например, «активный/пассивный»).
- Ключевая идея: даже относительно простые алгоритмы (как KNN) могут давать высокую точность при правильном выборе признаков и метрик расстояния.

Как отмечают авторы статьи об активности пользователей, несмотря на разнообразие поведения пользователей, метод может точно улавливать топологические особенности сети и эффективно классифицировать пользователей по уровню активности. Методы, основанные на машинном обучении, позволяют системам самостоятельно извлекать новые знания из обилия данных. За последние десятилетия многие статьи исследовали социальные сети через призму машинного обучения.

1.2.2. Метрики расстояния

Чтобы алгоритм мог определить, насколько два объекта похожи, необходимо уметь измерять расстояние между ними. Чем меньше расстояние между двумя точками в пространстве признаков, тем более похожи эти объекты.

Существует большое разнообразие метрик расстояния, одной из самых распространённых является евклидово расстояние.

Евклидово расстояние

Это расстояние по прямой между двумя точками. Для двух точек $A(x_1, x_2, \dots, x_n)$ и $B(y_1, y_2, \dots, y_n)$ оно вычисляется по формуле:

$$d = \sqrt{(x_1 - y_1)^2 + \dots + (x_n - y_n)^2}$$

Эта метрика часто используется для сравнения пользователей в многомерном пространстве признаков. Например, если у нас есть два пользователя с известными параметрами «время в сети» и «количество лайков», евклидово расстояние покажет, насколько они похожи. Чем меньше расстояние, тем более схоже их поведение.

Манхэттенское расстояние

Манхэттенское расстояние, также называемое расстоянием такси или расстоянием городского квартала, — это расстояние между двумя точками, которое равно сумме модулей разностей их координат. В отличие от евклидова расстояния, манхэттенское расстояние измеряет дистанцию не по кратчайшей прямой, а по блокам.

Формула манхэттенского расстояния:

$$d = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

В контексте анализа поведения пользователей манхэттенское расстояние может быть полезно, если признаки не связаны между собой и важны индивидуальные отклонения по каждой шкале. Например, при сравнении пользователей по таким показателям, как частота комментариев, количество репостов и время просмотра, эта метрика учитывает разницу по каждому признаку отдельно, не сглаживая её, в отличие от евклидова расстояния.

Расстояние Чебышёва

Расстояние Чебышёва — это метрика на векторном пространстве, которая определяется как максимум модуля разности соответствующих компонент двух векторов.

Формула расстояния Чебышёва:

$$d = \max(|x_1 - y_1|, |x_2 - y_2|, \dots, |x_n - y_n|)$$

Эта метрика ориентируется на самый «сильный» признак, по которому пользователи различаются больше всего. В задачах анализа поведения это может быть полезно, если один из поведенческих показателей (например, время в сети) значительно важнее остальных, и именно он определяет схожесть или различие пользователей.

Пример расчёта расстояний

Рассмотрим двух пользователей Instagram с тремя поведенческими признаками:

- Пользователь А: время в приложении 120 минут, 45 лайков, 3 комментария
- Пользователь Б: время в приложении 85 минут, 30 лайков, 1 комментарий

Рассчитаем расстояние между ними тремя способами.

Евклидово расстояние:

$$\begin{aligned} d &= \sqrt{(120 - 85)^2 + (45 - 30)^2 + (3 - 1)^2} \\ &= \sqrt{35^2 + 15^2 + 2^2} = \sqrt{1225 + 225 + 4} = \sqrt{1454} \approx 38,1 \end{aligned}$$

Манхэттенское расстояние:

$$d = |120 - 85| + |45 - 30| + |3 - 1| = 35 + 15 + 2 = 52$$

Расстояние Чебышёва:

$$d = \max(|120 - 85|, |45 - 30|, |3 - 1|) = \max(35, 15, 2) = 35$$

Чем меньше расстояние, тем больше пользователи похожи по поведению. В данном примере расстояние довольно большое, значит, пользователи ведут себя по-разному. Если бы оба ставили по 40 лайков и проводили по 100 минут, расстояние было бы близко к нулю.

Евклидово расстояние чувствительно к большим отклонениям по одному признаку, манхэттенское относится к отклонениям равномерно, а Чебышёва ориентируется на самый сильный признак.

1.2.3. Алгоритм К-ближайших соседей

Алгоритм KNN — это надежный и интуитивно понятный способ решения задач классификации и регрессии. Основной принцип алгоритма основан на нахождении К ближайших соседей на основе схожести их признаков.

Механизм К-ближайших соседей рассчитан на выявление расстояния между точками. При вводе новой точки запроса алгоритм выполняет поиск в сохраненном наборе данных, чтобы найти К обучающих выборок, наиболее близких к новому вводу.

- 1) Измерение расстояния: наиболее распространенной мерой является евклидово расстояние, которое измеряет расстояние по прямой между точками в многомерном пространстве.
- 2) Выбор соседей: после вычислений расстояний алгоритм сортирует их и определяет К ближайших записей.
- 3) Принятие решений:
 - Для классификации работает правило «Голосование большинством». Алгоритм смотрит на К ближайших соседей, определяет, к какому классу принадлежит каждый из них, и тот класс, который встречается чаще, побеждает.
 - Для регрессии действует правило «усреднение». Алгоритм смотрит на К ближайших соседей, определяет числовое значение у каждого из них, считает среднее арифметическое этих значений и принимает решение.

Преимущества и недостатки KNN

Как и любой алгоритм, KNN имеет свои сильные и слабые стороны.

Преимущества KNN

- Простота реализации и понимания. Алгоритм легко объяснить даже человеку, не знакомому с машинным обучением.
- Отсутствие этапа обучения. KNN не требует времени на обучение модели, так как просто запоминает все данные. За это свойство его называют «ленивым» алгоритмом.
- Универсальность. KNN может решать как задачи классификации, так и задачи регрессии.
- Устойчивость к выбросам. При правильном выборе параметра K влияние случайных выбросов снижается.

Недостатки KNN

- Медленная работа на больших данных. При каждом предсказании алгоритм вычисляет расстояния до всех объектов обучающей выборки, что требует много времени при большом объеме данных.
- Чувствительность к выбору K и метрики расстояния. Неправильный выбор параметров может существенно ухудшить качество предсказаний.
- Чувствительность к масштабу признаков. Если признаки имеют разные диапазоны значений, метрики расстояния могут искажаться. Поэтому перед применением KNN часто требуется нормализация данных.
- Проблема «проклятия размерности». При большом количестве признаков расстояния между объектами становятся почти одинаковыми, что снижает качество работы алгоритма.

1.2.4. Алгоритм K-Means

В машинном обучении существует также популярный алгоритм кластеризации — K-Means. Он относится к обучению без учителя: ему не нужны размеченные примеры. K-Means сам разбивает объекты на K групп (кластеров) так, чтобы объекты внутри одного кластера были максимально похожи, а объекты из разных кластеров — максимально различны.

В отличие от KNN, который предсказывает класс на основе известных данных, K-Means ищет скрытые группы там, где правильные ответы неизвестны.

Как работает алгоритм K-Means

Алгоритм работает итеративно и состоит из следующих шагов:

- 1) Выбор числа кластеров K — задаётся заранее (например, мы хотим разделить пользователей на 3 типа: пассивные, средние, активные).
- 2) Инициализация центров — случайным образом выбираются K точек, которые становятся центрами кластеров.
- 3) Распределение объектов — каждый объект (пользователь) относится к тому кластеру, центр которого находится ближе всего (расстояние обычно считается по евклидовой метрике).
- 4) Пересчёт центров — после того как все объекты распределены, центры кластеров пересчитываются как среднее арифметическое всех точек, входящих в кластер.
- 5) Повторение — шаги 3 и 4 повторяются до тех пор, пока центры кластеров не перестанут меняться, либо пока не будет выполнено заданное количество итераций.

Пример из жизни

Допустим, есть данные о пользователях: сколько времени они проводят в соцсети и сколько лайков ставят за день. Алгоритм K-Means может сам выделить группы:

- Кластер 0 (пассивные): мало времени, мало лайков;
- Кластер 1 (средние): среднее время, среднее количество лайков;
- Кластер 2 (активные): много времени, много лайков.

При этом алгоритму не нужно говорить заранее, кто к какому типу относится. Он сам находит эти группы на основе данных.

Преимущества K-Means

- Простота — алгоритм легко понять и реализовать.
- Быстрота — работает быстро даже на больших наборах данных.
- Масштабируемость — хорошо справляется с большим количеством объектов.

- Интерпретируемость — результаты кластеризации легко анализировать, так как каждый объект попадает ровно в один кластер.

Недостатки K-Means

- Нужно заранее знать K — число кластеров надо задавать вручную. Подбирается методом локтя или по силуэту (об этом в параграфе 1.5).
- Чувствительность к начальным центрам — если неудачно выбрать начальные центры, алгоритм может сойтись к плохому результату. Проблема решается запуском алгоритма несколько раз с разными начальными условиями.
- Чувствительность к выбросам — один резко отличающийся пользователь может сильно сместить центр кластера.
- Только выпуклые кластеры — алгоритм плохо работает, если кластеры имеют сложную форму (например, в виде полумесяца или кольца).

Применение K-Means к анализу поведения пользователей

В контексте данной работы K-Means используется для сегментации пользователей на основе их поведенческих признаков. Например, можно выделить группы пользователей по следующим параметрам:

- время, проведённое в соцсети;
- количество лайков;
- количество комментариев;
- частота публикаций.

Полученные кластеры затем интерпретируются (например, «активные», «средние», «пассивные») и используются для персонализации контента или выработки рекомендаций.

1.2.5. Методы оценки качества моделей

Чтобы оценить, насколько качественно модель справилась с поставленной задачей, используют специальные метрики. Для задач классификации (KNN) и кластеризации (K-Means) применяются разные подходы [?].

Метрики для классификации (KNN)

При оценке качества классификации обычно используют следующие метрики [?]:

- **Ассурасу (точность)** — это метрика, которая показывает долю правильно классифицированных объектов среди всех предсказанных. Другими словами, сколько объектов модель угадала верно из всех, что предсказала.
- **Precision (точность предсказания положительного класса)** — из всех объектов, которые модель отнесла к положительному классу (например, «активные пользователи»), сколько действительно являются таковыми.
- **Recall (полнота)** — из всех реально существующих объектов положительного класса, сколько модель смогла обнаружить.
- **F1-мера** — среднее гармоническое между Precision и Recall. Используется, когда нужно учесть обе метрики вместе.
- **Матрица ошибок (confusion matrix)** — таблица, показывающая количество правильных и ошибочных предсказаний. По диагонали располагаются верные ответы (TP и TN), а на побочной — ошибки (FP и FN).

TP (True Positive) — объекты правильно отнесённые к положительному классу. FN (False Negative) — объекты, которые модель ошибочно не отнесла к положительному классу. FP (False Positive) — объекты, которые модель ошибочно отнесла к положительному классу. TN (True Negative) — объекты правильно отнесённые к отрицательному классу [?].

Эти метрики дают полное представление о том, как модель справляется с задачей. Только точности (ассурасу) бывает недостаточно, если классы в данных сильно несбалансированы.

Метрики для кластеризации (K-Means)

Для оценки качества кластеризации, когда у нас нет правильных ответов, применяют внутренние метрики [?]:

- **Инерция (inertia)** — сумма квадратов расстояний от каждой точки до центра своего кластера. Чем меньше значение инерции, тем компактнее получились кластеры. Однако при увеличении числа кластеров инерция всегда уменьшается, поэтому её используют не как абсолютную меру, а для сравнения разных значений K.
- **Силуэт (silhouette score)** — показывает, насколько объект похож на свой кластер по сравнению с соседними. Значение силуэта лежит в диапазоне от -1 до 1. Чем ближе к 1, тем лучше: объект хорошо сгруппирован с «соседями» по кластеру и далёк от других кластеров. Значения около 0 означают, что кластеры пересекаются. Отрицательные значения говорят о том, что объект, вероятно, попал не в свой кластер.

В данной работе для оценки качества классификации KNN будут использоваться ассигасы, precision, recall и F1-мера, а также матрица ошибок. Для выбора оптимального числа кластеров K-Means будет применён метод локтя и оценка силуэта.

1.3. Краткая информация об инструментах разработки

Для написания кода и построения моделей в этой работе использовался язык Python. Он удобен для анализа данных и машинного обучения, потому что под него есть много готовых библиотек.

Основные библиотеки, которые я использовала:

- **pandas** — для работы с таблицами (загрузка, обработка, фильтрация данных).
- **numpy** — для математических операций и работы с массивами.
- **scikit-learn** — для реализации KNN и K-Means, а также для расчёта метрик (точность, силуэт, инерция).
- **matplotlib** — для построения графиков и визуализации кластеров.

Все эксперименты проводились в Google Colab — это среда, где можно писать и запускать Python-код прямо в браузере, без установки чего-либо на компьютер.

1.4. Описание источников данных

Для анализа был выбран датасет `instagram_usage_lifestyle.csv` с платформы Kaggle [?]. Он содержит поведенческие признаки 1 547 896 пользователей Instagram: время в приложении, лайки, комментарии, количество подписчиков и подписок, сессии, просмотры историй и другие метрики. Данные уже очищены от пропусков и пригодны для обучения.

На основе этого датасета решается задача кластеризации пользователей с помощью алгоритма K-Means для выделения поведенческих групп (активные, средние, пассивные пользователи).

2. Построение моделей машинного обучения и анализ результатов

2.1. Получение данных для построения моделей

Датасет был загружен из файла `instagram_usage_lifestyle.csv` с помощью библиотеки **pandas**. Первые строки данных были выведены на экран для

проверки корректности загрузки. Также была проверена размерность датасета и наличие пропущенных значений. Всего в выборке содержится 1 547 896 записей. Для ускорения вычислений была использована случайная выборка из 50 000 записей, что является репрезентативным объёмом для обучения модели.

2.2. Предобработка и исследовательский анализ данных

На данном этапе из датасета были отобраны числовые поведенческие признаки: время в приложении, количество лайков, комментариев, сообщений, подписчиков, подписок, сессий, постов, просмотров историй и рилсов, а также клики по рекламе.

Таблица 1. Описание поведенческих признаков

Признак	Диапазон	Описание
daily_active_minutes_instagram	0-480 мин	Время в приложении
likes_given_per_day	0-200	Лайки в день
comments_written_per_day	0-50	Комментарии в день
dms_sent_per_week	0-500	Отправленные сообщения
dms_received_per_week	0-500	Полученные сообщения
followers_count	0-10000	Подписчики
following_count	0-5000	Подписки
sessions_per_day	1-20	Сессии в день
posts_created_per_week	0-30	Посты в неделю
stories_viewed_per_day	0-100	Истории
reels_watched_per_day	0-150	Рилсы
ads_clicked_per_day	0-10	Клики по рекламе

Пропущенные значения были удалены, чтобы избежать искажения результатов. После этого признаки были приведены к единому виду с помощью библиотеки StandardScaler. Это необходимо, так как признаки имеют различные единицы измерения. Без стандартизации признаки с большими числовыми диапазонами могли бы доминировать над остальными.

2.3. Построение моделей

2.3.1. Кластеризация пользователей методом K-Means

Для кластеризации пользователей был выбран алгоритм K-Means. Необходимое число кластеров определялось методом локтя. График инерции показал, что после $K = 3$ падение инерции замедляется, поэтому было выбрано три кластера.

Как работает метод локтя

Метод локтя основан на анализе суммы квадратов расстояний от каждой точки до центра своего кластера. Эта сумма называется инерцией. Чем меньше инерция, тем компактнее получились кластеры. При увеличении числа кластеров инерция всегда уменьшается, потому что точки оказываются ближе к центрам. Однако после некоторого значения K уменьшение становится незначительным. На графике это выглядит как резкий изгиб — «локоть». Именно в этой точке и нужно выбирать число кластеров. В данном эксперименте инерция быстро падала при K от 1 до 3, а начиная с $K = 4$ снижение замедлилось. Это означает, что увеличение числа кластеров больше трёх не даёт значительного улучшения. Поэтому для анализа выбрано $K = 3$.

Модель K-Means была обучена на обработанных данных с параметрами: `n_clusters=3`, `random_state=42`, `n_init=10`. После обучения каждому пользователю был присвоен номер кластера (0, 1 или 2).

Размер полученных кластеров составил:

- Кластер 0: 15 713 пользователей (31.4%)
- Кластер 1: 15 877 пользователей (31.8%)
- Кластер 2: 18 410 пользователей (36.8%)

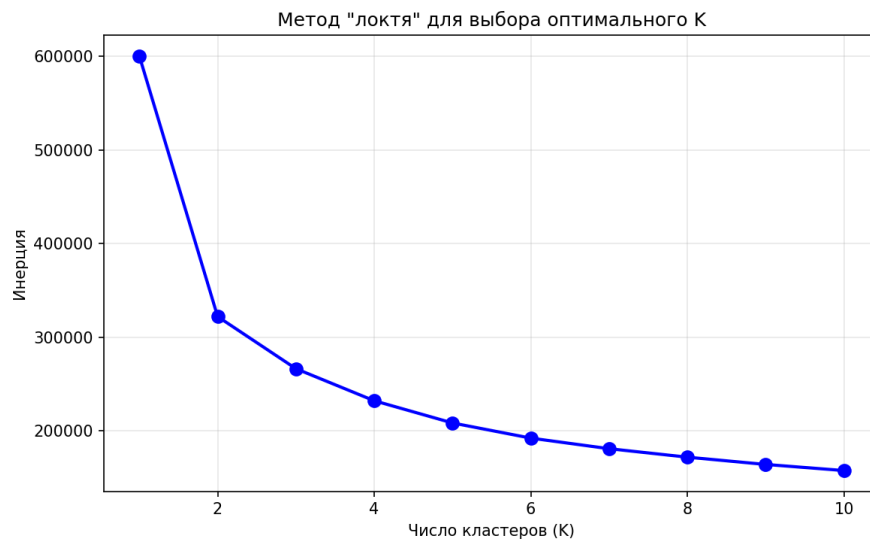


Рис. 1. Метод локтя для выбора числа кластеров

2.3.2. Классификация пользователей методом KNN

Для классификации пользователей по уровню активности был применён алгоритм K-ближайших соседей (KNN). Поскольку в исходном датасете не было готовых меток, они были созданы на основе индекса активности.

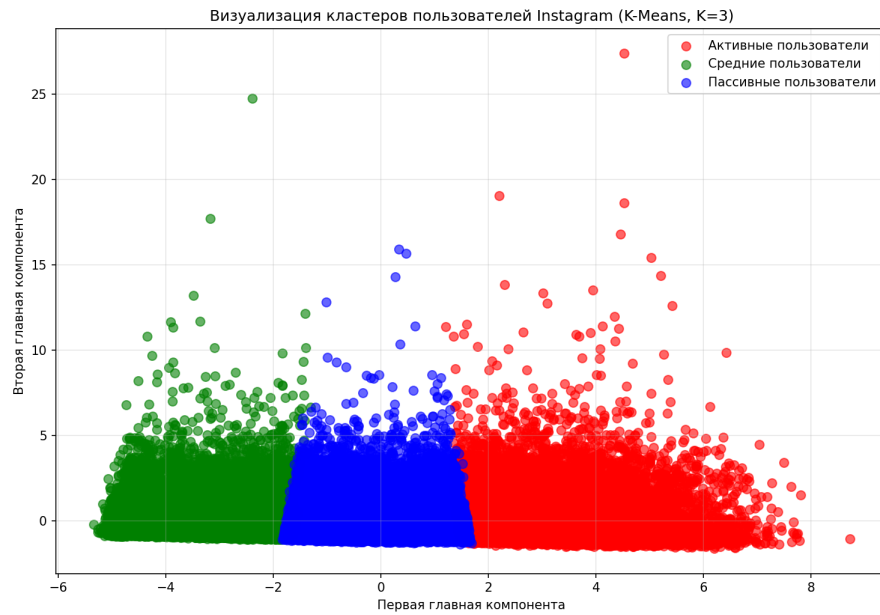


Рис. 2. Визуализация кластеров пользователей Instagram

Индекс активности рассчитывался по формуле:

$$\text{score} = \frac{\text{время в приложении}}{100} + \frac{\text{лайки}}{50} + \frac{\text{комментарии}}{10} + \text{посты} \times 10 + \text{сессии} \times 5$$

Затем все пользователи были разделены на три равные группы по процентиллям:

- пассивные ($\text{score} \leq 76.19$) — 33.0% пользователей;
- средние ($76.19 < \text{score} \leq 142.16$) — 33.0%;
- активные ($\text{score} > 142.16$) — 34.0%.

Данные были разделены на обучающую (80%, 40 000 объектов) и тестовую (20%, 10 000 объектов) выборки с сохранением пропорции классов.

2.4. Оценка качества моделей

2.4.1. Оценка качества кластеризации

Для оценки качества кластеризации был использован метод силуэта. Значение силуэта составило 0.2632. Это указывает на слабую или среднюю степень разделения кластеров. Поведение пользователей частично перекрывается, но общие группы различимы.

Таблица 2. Результаты кластеризации

Метрика	Значение	Интерпретация
Силуэт	0.2632	Слабая или средняя кластеризация
Количество кластеров	3	Оптимально по методу локтя
Объём выборки	50 000	Репрезентативная выборка

2.4.2. Оценка качества классификации

Для оценки качества классификации использовались метрики accuracy (точность), precision, recall и F1-мера. В таблице 3 представлены результаты для различных значений K .

Таблица 3. Результаты классификации KNN при различных значениях K

K	Accuracy	Precision	Recall	F1-score
3	0.9220	0.9221	0.9220	0.9220
5	0.9305	0.9305	0.9305	0.9304
7	0.9333	0.9333	0.9333	0.9332
9	0.9327	0.9328	0.9327	0.9326
11	0.9319	0.9320	0.9319	0.9318
15	0.9345	0.9346	0.9345	0.9344
21	0.9333	0.9334	0.9333	0.9332

Наилучшее качество классификации достигнуто при $K = 15$ с точностью 93.45%. Метрики precision, recall и F1-мера также находятся на уровне 0.934, что говорит о сбалансированной работе модели по всем трём классам.

Анализ матрицы ошибок показывает, что большинство пользователей классифицируются правильно. Ошибки возникают в основном на границе между соседними классами, что соответствует значению силуэта 0.2632.

2.4.3. Сравнение метрик расстояния

Было проведено сравнение трёх метрик расстояния при фиксированном $K = 15$.

Таблица 4. Сравнение метрик расстояния для KNN

Метрика	Accuracy	F1-score
Евклидова	0.9345	0.9344
Манхэттенская	0.9318	0.9317
Чебышёва	0.9251	0.9249

Евклидова метрика показала наилучший результат. Манхэттенская метрика даёт немного более низкую точность (93.18%), а метрика Чебышёва –

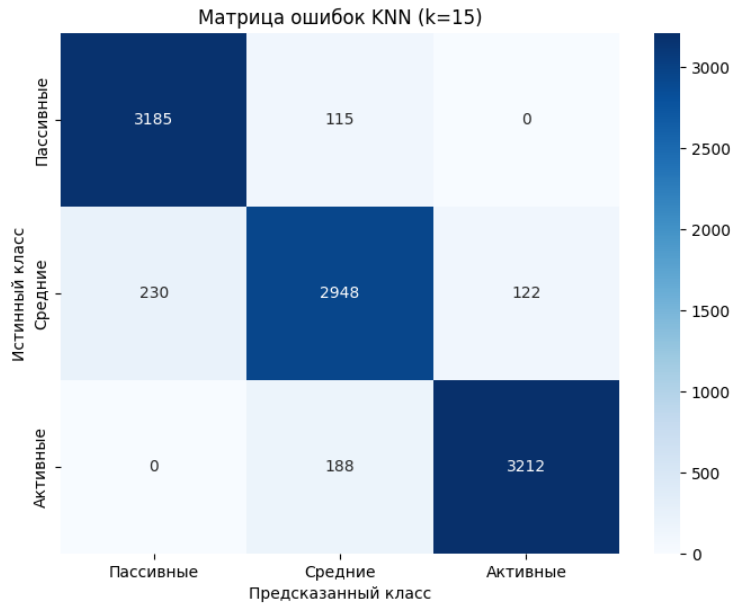


Рис. 3. Матрица ошибок классификации KNN (K=15)

92.51%. Это объясняется тем, что поведенческие признаки имеют непрерывную природу, и евклидово расстояние является для них естественной мерой близости.

2.5. Анализ результатов и рекомендации

На основе проведённых экспериментов можно сделать следующие выводы:

- Алгоритм K-Means успешно выделил три кластера пользователей, которые интерпретируются как пассивные (31.4%), средние (31.8%) и активные (36.8%). Значение силуэта 0.2632 указывает на то, что группы различимы, но имеют области пересечения.
- Алгоритм KNN показал высокую точность классификации (93.45% при K=15), что подтверждает возможность автоматического определения уровня активности пользователя на основе его поведенческих признаков.
- Евклидова метрика расстояния оказалась наиболее подходящей для данной задачи по сравнению с манхэттенской и метрикой Чебышёва.

На основе полученных кластеров можно сформулировать следующие рекомендации по персонализации контента:

- **Активным пользователям (36.8% аудитории):** рекомендуется предлагать разнообразный и сложный контент, включая длинные видео, серии постов, интерактивные элементы. Эти пользователи готовы тратить время на изучение контента и активно взаимодействуют с ним.
- **Пассивным пользователям (31.4% аудитории):** оптимальны короткие вовлекающие форматы (рилсы, истории, короткие видео). Для этой группы важно быстро привлечь внимание, так как их вовлечённость ниже.
- **Средним пользователям (31.8% аудитории):** рекомендуется смешанный контент с элементами как для активных, так и для пассивных пользователей, чтобы постепенно повышать их вовлечённость.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были решены все поставленные задачи.

- 1) Изучены теоретические основы анализа социальных сетей и методов машинного обучения: метрики расстояния (евклидова, манхэттенская, Чебышёва), алгоритмы KNN и K-Means, метрики оценки качества (силуэт, инерция, ассурасу, precision, recall, F1-мера).
- 2) Рассмотрены подходы к выделению поведенческих паттернов, определены ключевые признаки: лайки, комментарии, время в сети, частота публикаций, количество подписчиков.
- 3) Подобран и подготовлен датасет `instagram_usage_lifestyle.csv`, содержащий поведенческие признаки пользователей Instagram. Проведена предобработка и стандартизация данных. Для ускорения вычислений использована случайная выборка из 50 000 записей.
- 4) Реализован алгоритм K-Means для кластеризации пользователей. Модель выделила три кластера: пассивные (31.4%), средние (31.8%) и активные (36.8%). Значение силуэта составило 0.2632, что указывает на слабую или среднюю степень разделения кластеров.
- 5) Реализован алгоритм KNN для классификации пользователей. На основе индекса активности были созданы метки трёх классов. Наилучшее качество достигнуто при $K = 15$: Ассурасу = 93.45%, Precision = 0.9346, Recall = 0.9345, F1 = 0.9344.
- 6) Проведено сравнение метрик расстояния для KNN: евклидова показала лучший результат (93.45%), манхэттенская (93.18%), чебышёва (92.51%).
- 7) На основе полученных результатов сформулированы рекомендации по персонализации контента для каждой из трёх групп пользователей.

Перспективы дальнейшей работы: сравнение полученных результатов с другими алгоритмами классификации (логистическая регрессия, дерево решений, случайный лес), а также построение рекомендательной системы на основе выделенных кластеров.

ПРИЛОЖЕНИЕ

Листинг программного кода

```
# Курсовая работа
# Анализ поведенческих закономерностей пользователей социальных сетей
# Шуркалова А. Н.

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from collections import Counter

# 1. Загружаем данные

df = pd.read_csv('instagram_usage_lifestyle.csv')
print('Загружено', df.shape[0], 'строк,', df.shape[1], 'столбцов')

# Берём 50000 случайных пользователей
df = df.sample(n=50000, random_state=42)
print('Используем', len(df), 'записей')

# 2. Отбираем признаки

cols = [
    'daily_active_minutes_instagram',
    'likes_given_per_day',
    'comments_written_per_day',
    'dms_sent_per_week',
    'dms_received_per_week',
    'followers_count',
    'following_count',
    'sessions_per_day',
    'posts_created_per_week',
```

```

        'stories_viewed_per_day',
        'reels_watched_per_day',
        'ads_clicked_per_day'
    ]

X = df[cols].copy()
X = X.dropna()
print('Признаков:', len(cols))
print('Объектов после очистки:', len(X))

# 3. Стандартизация

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Метод локтя

inertia = []
for k in range(1, 11):
    m = KMeans(n_clusters=k, random_state=42, n_init=10)
    m.fit(X_scaled)
    inertia.append(m.inertia_)

plt.plot(range(1, 11), inertia, 'o-')
plt.xlabel('Число кластеров')
plt.ylabel('Инерция')
plt.title('Метод локтя')
plt.grid()
plt.savefig('elbow.png')
plt.show()

# 5. K-Means с 3 кластерами

model = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = model.fit_predict(X_scaled)

# 6. Размер кластеров

c = Counter(clusters)
print()
print('Распределение по кластерам:')
print('Кластер 0:', c[0], '(', round(c[0]/len(clusters)*100, 1), '%)')
```

```
print('Кластер 1:', c[1], '(', round(c[1]/len(clusters)*100, 1), '%)')
print('Кластер 2:', c[2], '(', round(c[2]/len(clusters)*100, 1), '%)')
```

7. Силуэт

```
sil = silhouette_score(X_scaled, clusters)
print()
print('Силуэт:', round(sil, 4))
```

8. Визуализация кластеров

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(10, 8))
colors = ['red', 'blue', 'green']
names = ['Пассивные', 'Средние', 'Активные']
for i in range(3):
    mask = (clusters == i)
    plt.scatter(X_pca[mask, 0], X_pca[mask, 1], c=colors[i], label=names[i])
plt.legend()
plt.title('Кластеры пользователей')
plt.savefig('clusters.png')
plt.show()
```

9. Создаём метки для KNN

```
df2 = df[cols].copy()
df2['score'] = (df2['daily_active_minutes_instagram'] / 100 +
               df2['likes_given_per_day'] / 50 +
               df2['comments_written_per_day'] / 10 +
               df2['posts_created_per_week'] * 10 +
               df2['sessions_per_day'] * 5)
```

```
q1 = df2['score'].quantile(0.33)
q2 = df2['score'].quantile(0.66)
```

```
def get_label(s):
    if s <= q1:
        return 0
    elif s <= q2:
        return 1
```



```

        else:
            return 2

df2['label'] = df2['score'].apply(get_label)
labels = df2['label'].values

print()
print('Метки для KNN:')
print('Пассивные:', sum(labels == 0), '(', round(sum(labels == 0)/len(la
print('Средние:', sum(labels == 1), '(', round(sum(labels == 1)/len(la
print('Активные:', sum(labels == 2), '(', round(sum(labels == 2)/len(l

# 10. Обучаем KNN

X_train, X_test, y_train, y_test = train_test_split(X_scaled, labels,

print()
print('Обучающая выборка:', len(X_train))
print('Тестовая выборка:', len(X_test))
print()
print('Результаты KNN:')
print('k    | Accuracy')
print('----|-----')

best_acc = 0
best_k = 0

for k in [3, 5, 7, 9, 11, 15, 21]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print(k, ' | ', round(acc, 4))
    if acc > best_acc:
        best_acc = acc
        best_k = k

print()
print('Лучшее k =', best_k, 'с точностью', round(best_acc*100, 2), '%')

# 11. Матрица ошибок

```

```

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train, y_train)
pred_best = best_knn.predict(X_test)

cm = confusion_matrix(y_test, pred_best)
print()
print('Матрица ошибок:')
print(cm)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Пассивные', 'Средние', 'Активные'],
            yticklabels=['Пассивные', 'Средние', 'Активные'])
plt.xlabel('Предсказано')
plt.ylabel('На самом деле')
plt.title('Матрица ошибок KNN')
plt.savefig('confusion_matrix.png')
plt.show()

```

12. Сравнение метрик расстояния

```

print()
print('Сравнение метрик:')
print('Метрика          | Точность')
print('-----|-----')

for m in ['euclidean', 'manhattan', 'chebyshev']:
    knn = KNeighborsClassifier(n_neighbors=best_k, metric=m)
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print(m, ' | ', round(acc, 4))

print()
print('Готово')

```